



Projektportfolio

IU Internationale Hochschule

Studiengang: M.Sc. Informatik

Architekturdokument

Projekt Software Engineering

DLMCSPSE01_D_CF

GitHub: <https://github.com/micael92/FlowWorkbench>

Download: <https://github.com/micael92/FlowWorkbench/releases/tag/v0.1.0>

Michael Hilfer

Matrikelnummer: 3205669

Tutor: Prof. Dr. Dirk Simon

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	IV
1. Technologieübersicht.....	1
1.1 Programmiersprache und Laufzeitumgebung	1
1.2 Frameworks und Bibliotheken.....	1
2. Architektur	3
2.1 Architekturentwurf.....	3
2.2 Zentrale Klassen und ihre Zusammenarbeit	4
2.3 Exemplarischer Systemablauf	7
Literaturverzeichnis.....	9

Abbildungsverzeichnis

Abb. 1: Architekturentwurf der Anwendung	4
Abb. 2: UML-Klassendiagramm mit den wichtigsten Attributen und Methoden	6
Abb. 3: Sequenzdiagramm zum Ablauf der Entfernung eines Features.	8

Tabellenverzeichnis

Tab. 1: Übersicht der in FlowWorkbench eingesetzten Frameworks und Bibliotheken.	2
---	---

1. Technologieübersicht

1.1 Programmiersprache und Laufzeitumgebung

Für die Umsetzung des Projekts wurde mit Python 3 (Version 3.12) eine moderne objektorientierte Hochsprache gewählt. Python ist in der Datenanalyse, Datenverarbeitung und im Bereich des maschinellen Lernens weit verbreitet und bietet damit ein großes Ökosystem mit zahlreichen etablierten Bibliotheken, die für die Anwendungszwecke von FlowWorkbench genutzt werden können. Etablierte Bibliotheken wie beispielsweise pandas für die Datenverarbeitung oder matplotlib für Datenvisualisierungen können mit geringer Komplexität in eine Python-basierte Anwendung integriert werden, bieten aber ein umfangreiches Ausmaß an verlässlichen Funktionalitäten. Zudem sind Python und viele der eingesetzten Bibliotheken im Zielumfeld der Anwendung weit verbreitet, sodass die quelloffene Anwendung in Zukunft von Nutzern weiterentwickelt oder angepasst werden kann. Neben der Implementierung der Anwendung in Python wurden auch andere Programmiersprachen wie beispielsweise Java oder C# in Betracht gezogen, wobei in deren Nutzung keine ausschlaggebenden Vorteile gesehen wurden, die ein Verlassen des Data-Science-Ökosystems von Python rechtfertigen würden.

Die Anwendung kann als reine Desktopanwendung grundsätzlich über einen lokalen Python-Interpreter auf dem Zielsystem ausgeführt werden. Zur Ausführung sind demnach weder eine Server- oder Webverbindung noch ein Browser notwendig, sodass die Anwendung wie in den Anforderungen festgelegt lokal auf einem Windows-10/11-Betriebssystem ausgeführt werden kann.

1.2 Frameworks und Bibliotheken

Zur Implementierung von FlowWorkbench wurden die in Tabelle 1 dargestellten Frameworks und Bibliotheken eingesetzt. Die Nutzung von Frameworks und Bibliotheken wurde der kompletten Eigenentwicklung aller Anwendungsteile vorgezogen, weil so auf bereits vorhandene Technologien zurückgegriffen werden konnte, die mit den im Projekt vorhandenen Entwicklungskapazitäten nicht umsetzbar gewesen wären. Des Weiteren weisen die gewählten Frameworks und Bibliotheken aufgrund ihrer langjährigen Entwicklung und breiten Nutzung einen hohen Grad an Erprobung und technischer Reife auf, der im Rahmen einer Eigenentwicklung nicht erreichbar gewesen wäre.

Bei der Auswahl der Frameworks und Bibliotheken wurden verschiedene Aspekte einbezogen. Grundsätzlich mussten die in Betracht gezogenen Frameworks und Bibliotheken eine Lizenz aufweisen, die eine Nutzung in einer Open-Source-Anwendung ermöglicht. Zusätzlich wurde Wert darauf gelegt, möglichst etablierte und aktiv gepflegte Bibliotheken zu nutzen, um auf erprobte Funktionen und eine ausreichende Dokumentation zurückgreifen zu können.

Für die grafische Benutzeroberfläche wird auf das plattformübergreifende Qt-Framework zurückgegriffen. Die Funktionalitäten des Frameworks können dabei über PySide6 innerhalb der Python-Anwendung genutzt werden. Das Framework bietet verschiedene Komponenten, die zum Aufbau einer

Windows-Desktopanwendung notwendig sind, und unterstützt dabei grundsätzlich den Ansatz der Trennung der grafischen Oberfläche von den zugrundeliegenden Daten der Anwendung durch eine Model/View-Architektur (*Qt Documentation*, 2026). Dieser Architekturansatz passt somit zur geplanten Anwendungsarchitektur von FlowWorkbench, da auch hier eine Trennung der grafischen Oberfläche von anderen Anwendungsteilen angestrebt wird.

Zur Verarbeitung der Datensätze wird auf die verbreitete Bibliothek pandas gesetzt. Diese bietet mit der Datenstruktur des DataFrame umfangreiche Möglichkeiten zur Verarbeitung und Analyse von strukturierten Daten, wie den für FlowWorkbench vorgesehenen IDS-Daten im CSV-Format (McKinney, 2010, S. 1–2). Die pandas-Bibliothek ist damit neben dem PySide6-Framework eine zentrale Komponente der zu entwickelnden Anwendung FlowWorkbench. Ergänzend hierzu wird die Matplotlib-Bibliothek eingesetzt, um einzelne Visualisierungsanforderungen in der Anwendung umzusetzen. Zusätzlich wird das Testframework pytest eingesetzt, um entwicklungsbegleitende Tests automatisiert durchzuführen.

Tab. 1: Übersicht der in FlowWorkbench eingesetzten Frameworks und Bibliotheken.

Bibliothek/ Framework	Zweck in der Anwen- dung	Begründung
PySide6	GUI	Qt-basiertes Desktop-Framework mit benötigten GUI-Komponenten
pandas	Verarbeitung des Datensatzes	Etablierte DataFrame-Strukturen und umfangreiche Funktionen für tabellarische Daten
Matplotlib	Visualisierung der Labelverteilung	Etablierte Bibliothek für benötigte Datenvisualisierungen
pytest	Automatisierte Tests	Reproduzierbare entwicklungsbegleitende Tests

Quelle: Eigene Darstellung.

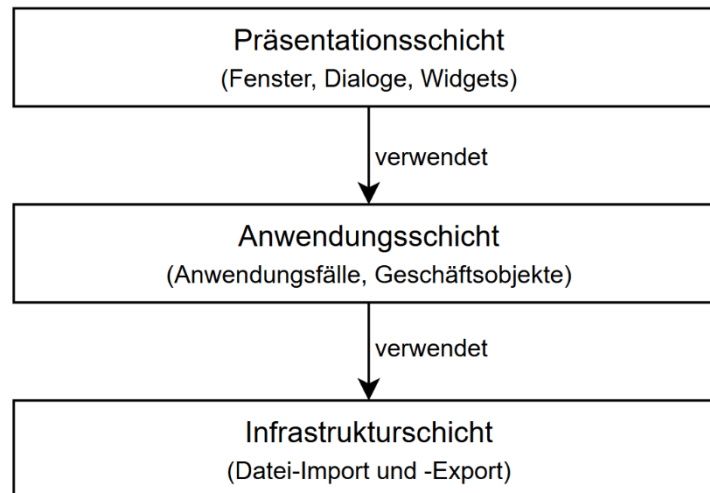
2. Architektur

2.1 Architekturentwurf

Bei der Auswahl der Architektur für die zu entwickelnde Anwendung wurden im Schwerpunkt die von Starke in seinem Werk „Effiziente Softwarearchitekturen“ (2024, S. 88–123) beschriebenen Architekturmuster in Betracht gezogen. Starke beschreibt ein breites Feld an Architekturmustern und bietet damit eine solide Grundlage zur Auswahl einer Softwarearchitektur für die zu entwickelnde Anwendung im gegebenen Projektumfang. Aufgrund der Konzeption der Anwendung als lokale Desktopanwendung wurden aus den aufgeführten Architekturmustern die Schichten-, Ports- und Adapter, sowie die Model-View-Controller-Architektur in Betracht gezogen. Nach einem Vergleich der drei genannten Architekturmuster fiel die Architekturentscheidung für FlowWorkbench auf eine Schichtenarchitektur.

Die Schichtenarchitektur ermöglicht eine flexible Strukturierung der Anwendung durch anwendungsspezifische Schichten. Typischerweise bietet dabei eine Schicht einer übergeordneten Schicht Dienste zur Nutzung an, wobei ihre inneren Eigenschaften von außen nicht sichtbar sind. Das Architekturmuster fördert damit, dass Abhängigkeiten zwischen den Schichten strukturiert implementiert werden, und bietet zudem die Möglichkeit, einzelne Schichten bei Bedarf mit begrenzten Auswirkungen auszutauschen. Als nachteilig können in einer Schichtenarchitektur Performance-Einbußen gesehen werden, die entstehen, wenn eine Schicht für die Nutzung einer anderen Schicht zunächst eine oder mehrere Zwischenschichten konsultieren muss. Für derartige Fälle kann jedoch auch auf ein Layer-Bridging zurückgegriffen werden, welches den direkten Zugriff auf eine entferntere Schicht beschreibt (Starke, 2024, S. 88–89). Die Schichtenarchitektur wurde insbesondere gewählt, um in der zu entwickelnden Anwendung eine strukturierende Trennung zwischen den Programmteilen der grafischen Benutzeroberfläche, den logischen Anwendungsfällen sowie den technischen Komponenten zur Interaktion mit der Infrastruktur zu schaffen. FlowWorkbench wurde deswegen in die in Abbildung 1 dargestellten Schichten aufgeteilt. Die Schichten sind nach ihren Verantwortungsbereichen aufgeteilt, sodass jede Schicht einen Programmteil abdeckt. In der gewählten Schichtenarchitektur ist eine grundsätzliche Abhängigkeitsrichtung vorgesehen, in der die Präsentationsschicht von der Anwendungsschicht abhängt, wobei diese wiederum von der Infrastrukturschicht abhängt. Ein Zugriff aus der Präsentationsschicht auf die Infrastrukturschicht ist nicht vorgesehen. Im aktuellen Entwicklungsstand der Anwendung konnte somit auf die zuvor dargestellte Möglichkeit des Layer-Bridgings verzichtet werden. Auch Abhängigkeiten einer unteren Schicht von einer darüberliegenden Schicht sind nicht vorgesehen. So werden Abhängigkeitskreise unterbunden und einzelne Schichten können einfacher verändert werden.

Abb. 1: Architekturentwurf der Anwendung



Quelle: Eigene Darstellung.

In der Präsentationsschicht werden alle Komponenten verortet, die zur Darstellung der Anwendungsdaten in der grafischen Oberfläche und zur Benutzerinteraktion benötigt werden, sodass hier hauptsächlich die notwendigen Teile des PySide6-Frameworks eingesetzt werden.

In der Anwendungsschicht werden alle Komponenten vorgehalten, die einen Anwendungsfall oder fachliche Komponenten wie den importierten Datensatz, abbilden. An dieser Stelle wurde auf eine in einem frühen Architekturentwurf vorgesehene Domänenschicht zur Verortung der domänen- und fachspezifischen Programmteile verzichtet. Während der Implementierung wurde deutlich, dass eine zusätzliche Domänenschicht zusätzliche Abstraktionen und Abhängigkeiten erfordert hätte, ohne dabei einen erkennbaren Vorteil zu liefern. Die fachlichen Komponenten konnten stattdessen sinnvoll der Anwendungsschicht zugeordnet werden.

In der Infrastrukturschicht wurden Komponenten zur Interaktion mit dem Dateisystem verortet. Im aktuellen Entwicklungsstand der Anwendung werden hier insbesondere das Einlesen und Schreiben von CSV-Datensätzen umgesetzt. Bei einer Weiterentwicklung der Anwendung sollten Programmteile zum Import und Export anderer Dateiformate oder andere technische Schnittstellen zur Kommunikation hier verortet werden. Auch Ansätze zur technischen Verarbeitung größerer Datensätze, wie z.B. das Lesen von Datensätzen in mehreren Teilen sollten in der Infrastrukturschicht implementiert werden.

2.2 Zentrale Klassen und ihre Zusammenarbeit

Die konkrete Klassenstruktur mit den relevantesten Klassen ist in Abbildung 2 in Form eines UML-Klassendiagramms dargestellt. Im Folgenden sollen die relevantesten Klassen und insbesondere ihre Abhängigkeiten beschrieben werden, sodass die grundlegende Funktionsweise der Anwendung und die Umsetzung der eingangs erläuterten Schichtenarchitektur verdeutlicht werden können.

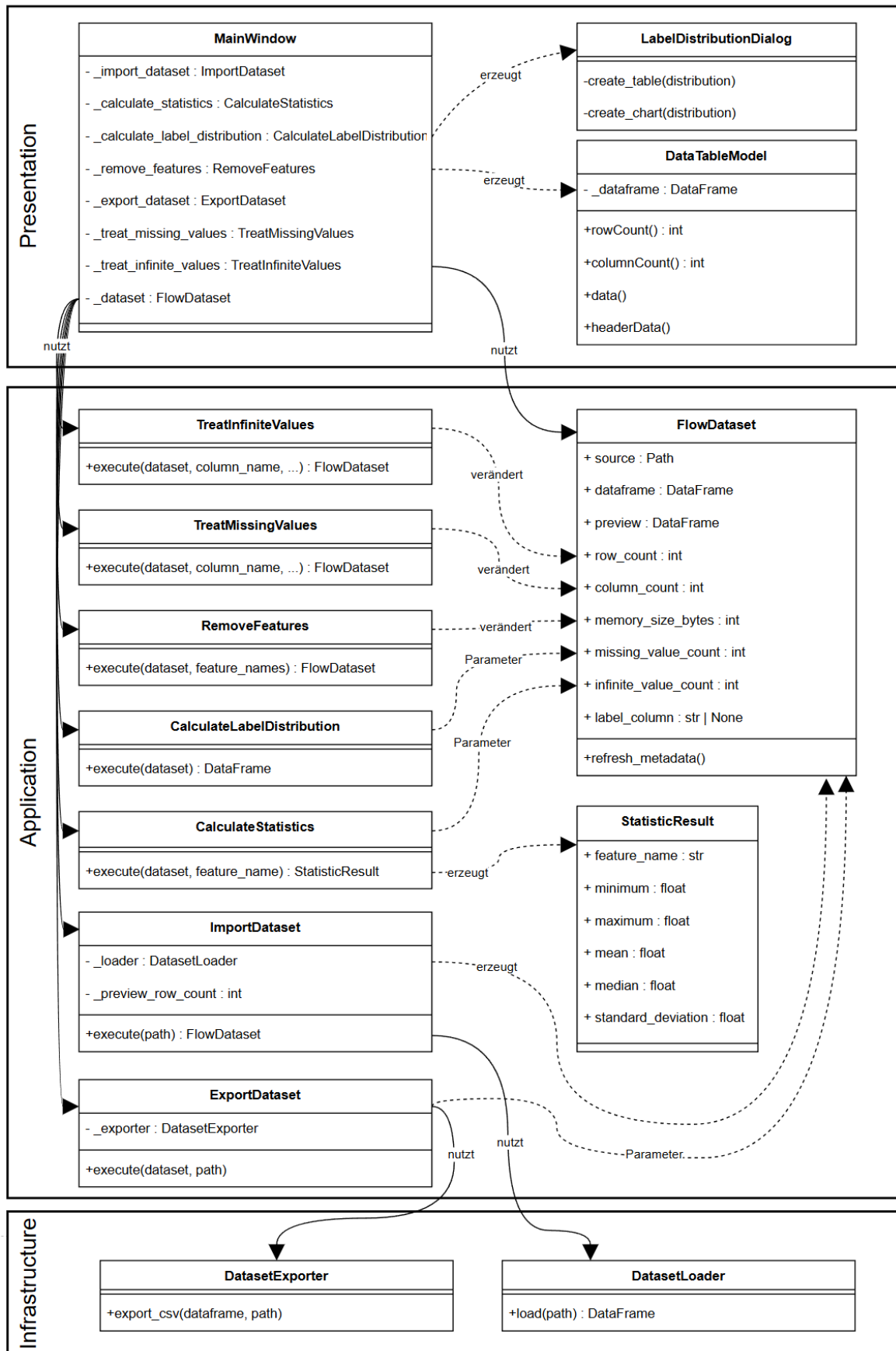
In der Präsentationsschicht ist die Klasse `MainWindow` die zentrale Klasse. Sie hält eine Referenz auf den importierten Datensatz in Form einer Instanz der Klasse `FlowDataset` und löst auf Basis von Benutzerinteraktionen die verschiedenen Anwendungsfälle aus der Anwendungsschicht aus. Dazu verwaltet `MainWindow` die notwendigen GUI-Elemente wie das Fenster, die Buttons, Dialoge oder die tabellarische Vorschau des Datensatzes. Für den Dialog zur Darstellung der Labelverteilung und für die tabellarische Vorschau des Datensatzes werden von `MainWindow` Instanzen der Klassen `LabelDistributionDialog` und `DataTableModel` erzeugt.

In der Anwendungsschicht ist die Klasse `FlowDataset` das zentrale Element. Eine `FlowDataset`-Instanz wird nach der Ausführung des Importvorgangs durch die Klasse `ImportDataset` erzeugt und hält eine Referenz auf einen `DataFrame` mit den importierten Daten. Zusätzlich werden in den Attributen der `FlowDataset`-Instanz relevante Metadaten des Datensatzes vorgehalten. So werden hier beispielsweise die Zeilen- und Spaltenanzahl, die Anzahl der unendlichen und fehlenden Werte oder die Bezeichnung der Label-Spalte hinterlegt. Zusätzlich verfügt die Klasse mit `refresh_metadata` über eine Methode zur Aktualisierung dieser Metadaten, sodass nach Änderungen am Datensatz die Attribute des Objekts aktualisiert werden können.

Die Anwendungsfälle sind in eigenen Klassen definiert. Sie lassen sich grundsätzlich in solche unterscheiden, die den geladenen Datensatz bzw. den `DataFrame` verändern, und solche, die den Datensatz lediglich als Parameter nutzen. So verändern die Klassen aus der fachlichen Datenvorverarbeitung, wie das Behandeln von fehlenden und unendlichen Werten oder das Entfernen von Features, den geladenen Datensatz im Arbeitsspeicher. Die Klassen zur Berechnung von Statistiken, der Labelverteilung oder zum Export nutzen den Datensatz dagegen als Parameter für die auszuführenden Operationen und verändern ihn nicht. Die Ergebnisse einer statistischen Berechnung werden durch die Klasse `StatisticResult` abgebildet, deren Instanzen durch `CalculateStatistics` erzeugt werden.

Die Klassen `ImportDataset` und `ExportDataset` bilden die Anwendungsfälle des Imports und Exports ab, wobei sie hierzu auf entsprechende Klassen in der Infrastrukturschicht zurückgreifen. In der Infrastrukturschicht kapseln die Klassen `DatasetLoader` und `DatasetExporter` die technischen Funktionen zum Lesen und Schreiben eines Datensatzes im CSV-Format aus bzw. in dem Dateisystem des Betriebssystems.

Abb. 2: UML-Klassendiagramm mit den wichtigsten Attributen und Methoden



Quelle: Eigene Darstellung.

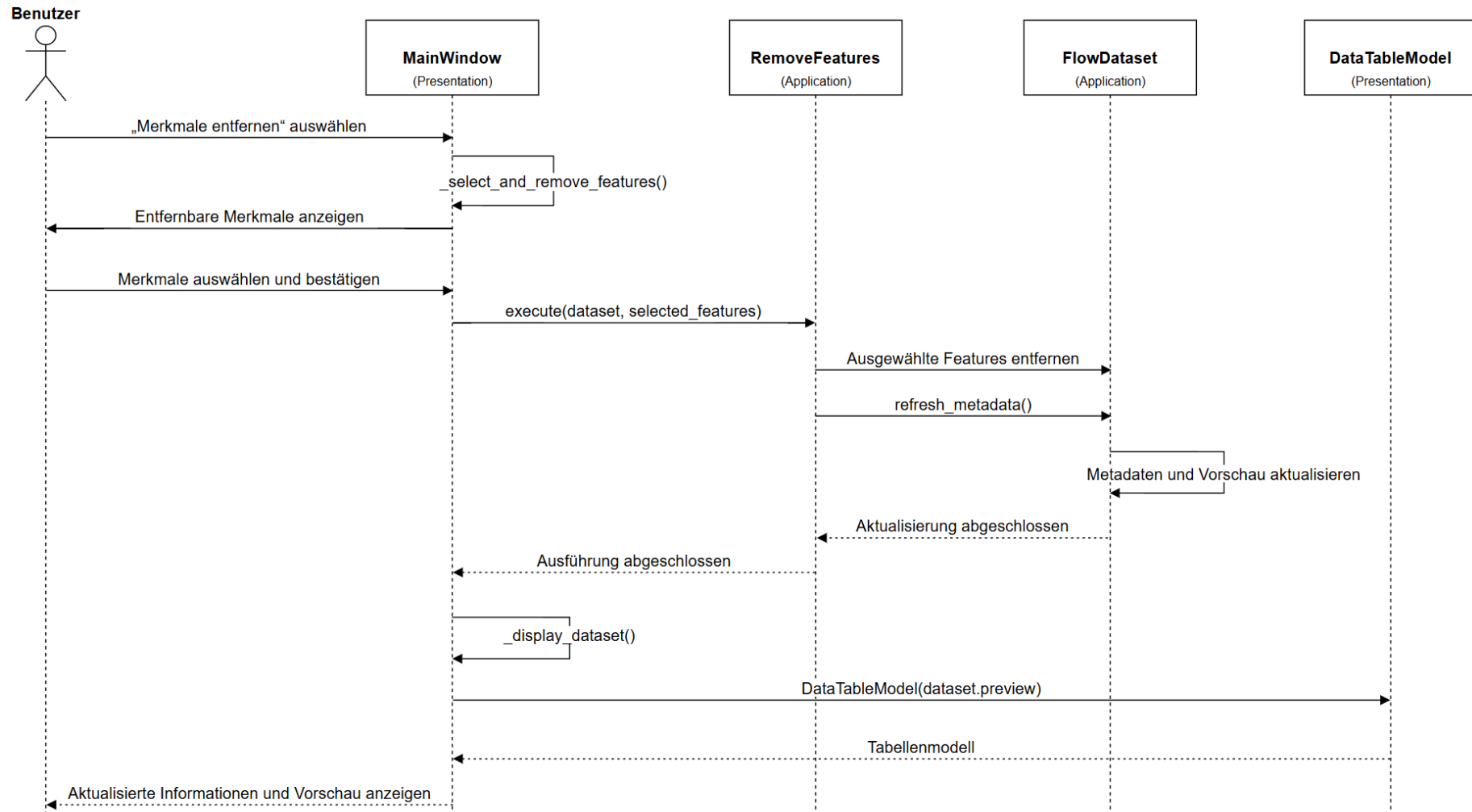
2.3 Exemplarischer Systemablauf

Als Beispiel für einen zentralen Systemablauf soll der Anwendungsfall des Entfernens eines Features dargestellt werden. Der Ablauf ist dazu in Abbildung 3 in einem UML-Sequenzdiagramm visualisiert.

Das Entfernen eines Features aus einem Datensatz ist nur möglich, sofern ein Datensatz in die Anwendung importiert wurde. Ist dies nicht der Fall, bleibt die Schaltfläche zum Auslösen des Dialogs gesperrt. Bei geladenem Datensatz wird der Ablauf damit durch die Auswahl der entsprechenden Schaltfläche in der grafischen Oberfläche der Anwendung gestartet. Das Auslösen der Schaltfläche „Merkmale entfernen“ löst auf Ebene der Präsentationsschicht in MainWindow die Methode `_select_and_remove_features()` aus, welche die entfernbaren Merkmale ermittelt und dem Benutzer in einem Auswahldialog anbietet. Abgesehen von der Label-Spalte sind grundsätzlich alle Merkmale entfernbar. Der Benutzer wählt anschließend die zu entfernenden Merkmale und bestätigt diese. Daraufhin wird die `execute`-Funktion von `RemoveFeatures` aufgerufen. Dabei werden Verweise auf den Datensatz und die zu entfernenden Merkmale übergeben. Durch diesen Aufruf wird die Bearbeitung des Datensatzes in der Anwendungsschicht ausgelöst. Die Funktion greift auf das übergebene `FlowDataset` zu und entfernt die gewählten Features. Anschließend werden mit der `refresh_metadata`-Methode die Eigenschaften des `FlowDataset` neu ermittelt, da Attribute wie die Spaltenanzahl geändert wurden. Dabei wird insbesondere auch das `preview`-Attribut aktualisiert, in dem die Daten für die grafische Tabellenvorschau vorgehalten werden. Nach Abschluss der Feature-Entfernung und Aktualisierung der Attribute von `FlowDataset` wird in die Präsentationsschicht zurückgekehrt und in MainWindow die Methode `_display_dataset()` aufgerufen, um die Tabellenvorschau in aktualisierter Fassung anzuzeigen. Dazu wird ein neues `DataTableModel` mit dem zuvor aktualisierten `preview`-Attribut erzeugt, sodass anschließend in der GUI sowohl die aktualisierten Datensatzinformationen als auch die aktualisierte Datensatzvorschau angezeigt werden.

Im Sequenzdiagramm wird so ein grundlegendes Ablaufmuster deutlich, das auch in anderen Anwendungsfällen genutzt wird. In der grafischen Oberfläche auf der Präsentationsschicht wird ein Anwendungsfall durch eine Benutzerinteraktion ausgelöst. Die Präsentationsschicht löst anschließend einen Anwendungsfall in der Anwendungsschicht aus, welcher die entsprechende Verarbeitung koordiniert und abhängig vom jeweiligen Anwendungsfall Daten auswertet oder verändert. Anschließend wird über die Präsentationsschicht die grafische Oberfläche aktualisiert und dem Benutzer bereitgestellt. Bei Anwendungsfällen zum Import und Export greift die Anwendungsschicht zusätzlich auf die Infrastrukturschicht zu.

Abb. 3: Sequenzdiagramm zum Ablauf der Entfernung eines Features.



Quelle: Eigene Darstellung.

Literaturverzeichnis

McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>

Qt Documentation. (2026). Qt Python Binding Model/View Programming. <https://doc.qt.io/qt-6/model-view-programming.html>

Starke, G. (2024). Effektive Softwarearchitekturen. In *Effektive Softwarearchitekturen* (S. I–X). Carl Hanser Verlag GmbH & Co. KG. <https://doi.org/10.3139/9783446479098.fm>